

Covid-19 Data Integration, Analysis and Visualization Platform

Accentura Data Engineering Bootcamp Project Report

Student name: Mark Tretiak

Date: July 27, 2026

GitHub repository: <https://github.com/MarkTretiak/covid19-platform.git>

Table of Contents

1. Introduction	3
1.1 Architecture.....	3
2. Task 1 – Setup Snowflake and Resource Monitor	4
3. Task 2 – Data Exploration and Enhancement.....	5
3.1 Exploration.....	5
3.2 Enhancement	5
3.3 Automated EDA.....	6
4. Task 3 – NoSQL Data Modeling	7
4.1 Schema Design	7
5. Task 4 – API Development with Python	8
6. Task 5 – Interactive Visualization with Python.....	11
6.1 Bonus: User Annotations	11
7. Task 6 – Analytical Features	13
7.1 Time Series Forecasting.....	13
7.2 Bonus: Clustering.....	13
8. Task 7 – Performance Optimization	15
9. Task 8 – API Caching	16
10. Task 9 – Pattern Recognition (MATCH_RECOGNIZE)	17
10.1 Cumulative Rise Streaks	17
10.2 Accelerating New-Case Bursts.....	17
11. Task 10 – Source Code and Repository.....	19
12. Key Insights	19
13. Limitations	19
14. Conclusion.....	20

1. Introduction

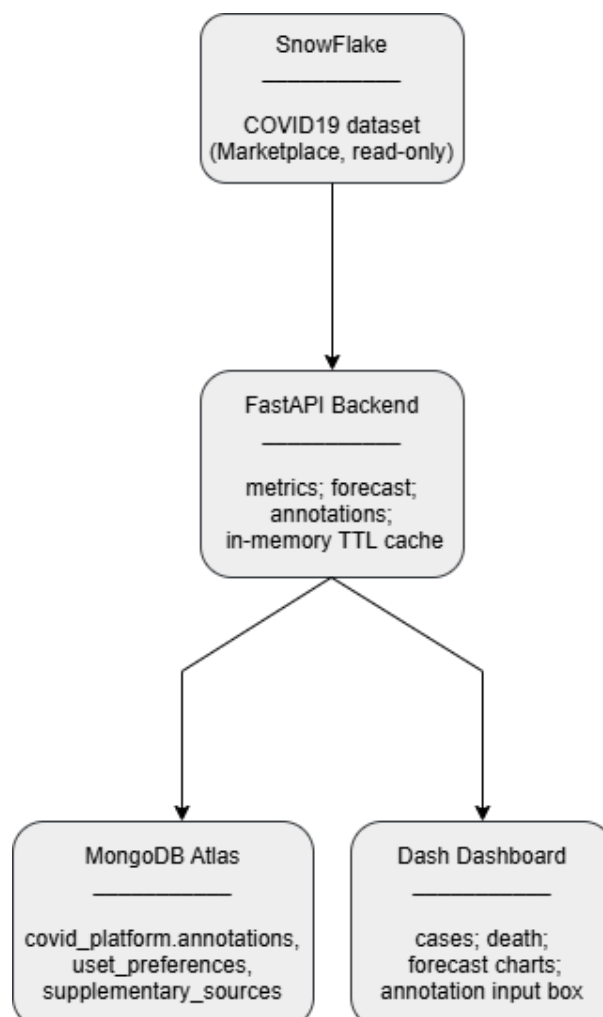
In this report is a design, implementation and findings of the COVID-19 Data Integration, Analysis and Visualization Platform built for the bootcamp project. The platform integrates a data warehouse from Snowflake, a document store from MongoDB, a python API layer from FastAPI, and interactive dashboard using Dash/Plotly, and also analytical features, such as time series forecasting, clustering and SQL pattern recognition.

For the project we are using free “COVID-19 Epidemiological Data dataset” from the Snowflake Marketplace, enriched with country population data from Kaggle to compute per-capita metrics.

1.1 Architecture

The project has this data flow:

- Snowflake Marketplace dataset (read-only, shared) → queried and pivoted into a flat structure
- Enriched with population data (Kaggle) → written to a project-owned Snowflake table, COVID_ENRICHED
- FastAPI backend queries COVID_ENRICHED and a MongoDB Atlas cluster, with in-memory caching
- Dash dashboard consumes the API to render interactive charts, a forecast, and user annotations



2. Task 1 – Setup Snowflake and Resource Monitor

We are creating a free Snowflake trial account on AWS cloud, Stockholm (eu-north-1) region. The “COVID-19 Epidemiological Data” dataset (published by Starschema, sourced from Johns Hopkins University) was added from the Snowflake Marketplace under Data Products, which mounts it as a read-only shared database, COVID19_EPIDEMIOLOGICAL_DATA.

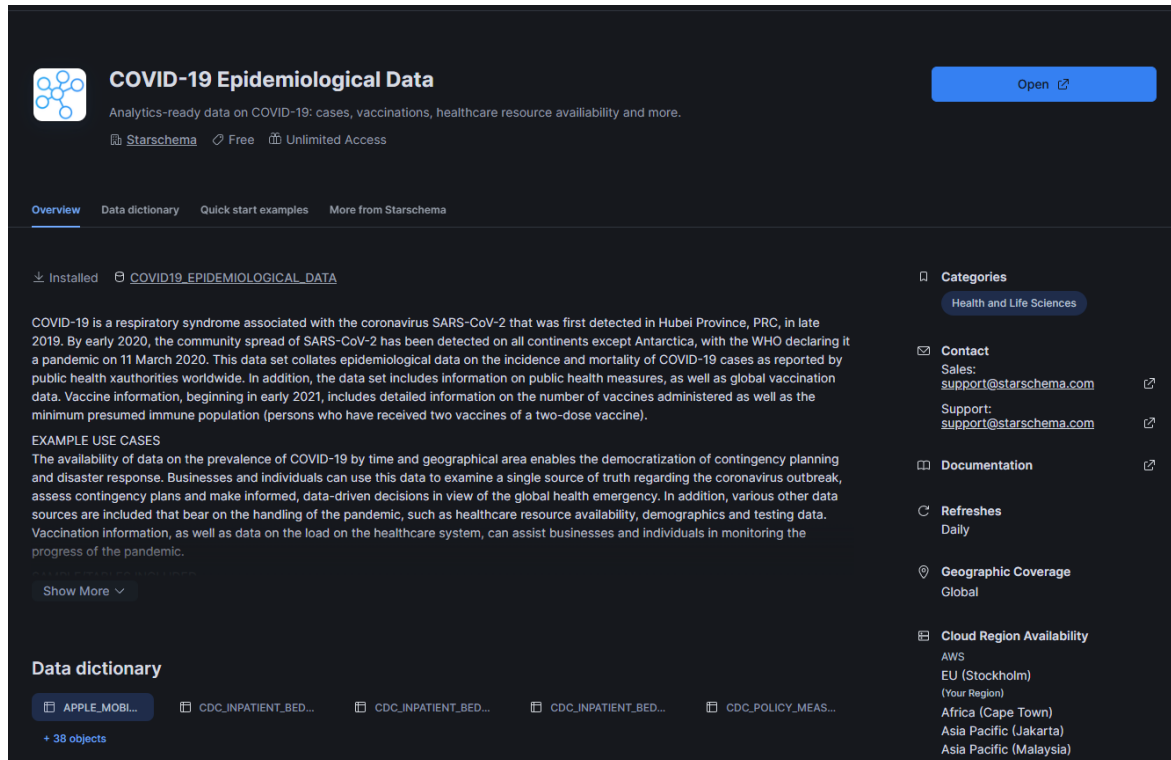


Figure 2.1 – COVID19 Epidemiological Data

A Resource Monitor named “covid_monitor” was configured with a 25-credit monthly quota and staged notification/suspension thresholds at 75%, 90%, and 100% of quota, to protect the trial account's limited credits from being exhausted by accident.

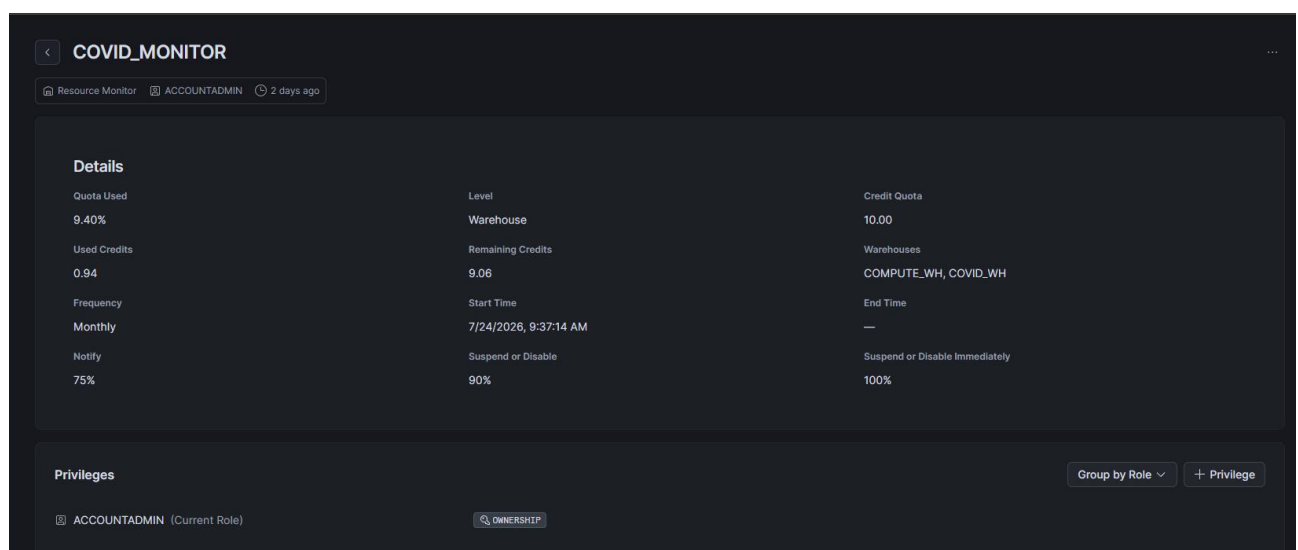
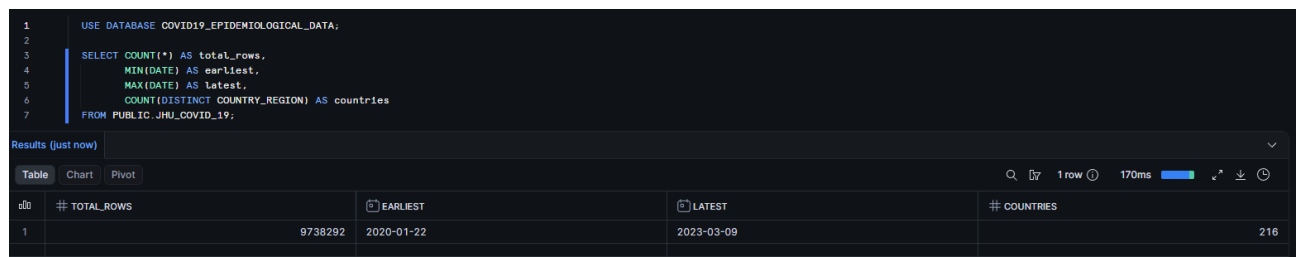


Figure 2.2 Resource Monitor

3. Task 2 – Data Exploration and Enhancement

3.1 Exploration

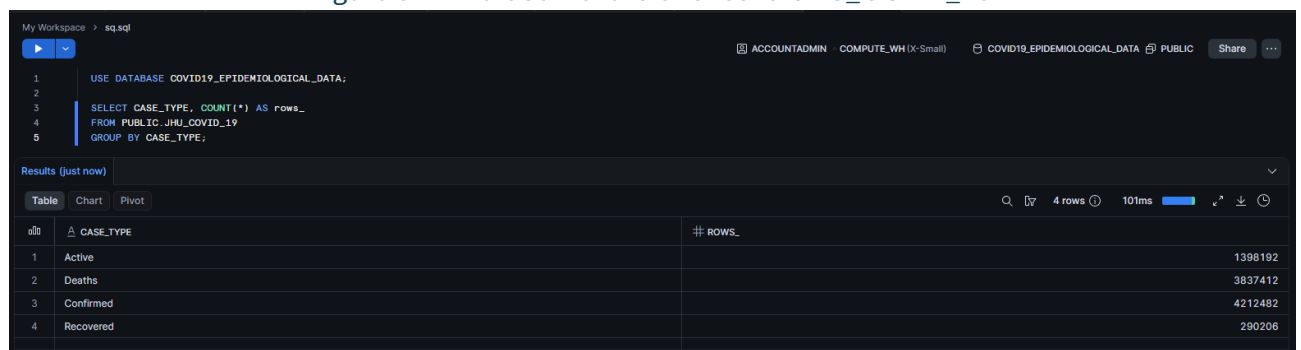
SQL exploration of JHU_COVID_19 shows that table uses a long/pivoted schema where each row represents one combination, such as country; date or case_type, also where case_type takes values: “confirmed”, “deaths”, “recovered” or “active”, and “cases” holds the count for that type. There is no separate “deaths” column – all four metrics live in the single “cases” column, disambiguated by “case_type”.



```
1 USE DATABASE COVID19_EPIDEMIOLOGICAL_DATA;
2
3 SELECT COUNT(*) AS total_rows,
4        MIN(DATE) AS earliest,
5        MAX(DATE) AS latest,
6        COUNT(DISTINCT COUNTRY_REGION) AS countries
7 FROM PUBLIC.JHU_COVID_19;
```

	# TOTAL_ROWS	EARLIEST	LATEST	# COUNTRIES
1	9738292	2020-01-22	2023-03-09	216

Figure 3.1 – Dataset-level statistics for JHU_COVID_19



```
1 USE DATABASE COVID19_EPIDEMIOLOGICAL_DATA;
2
3 SELECT CASE_TYPE, COUNT(*) AS rows_
4 FROM PUBLIC.JHU_COVID_19
5 GROUP BY CASE_TYPE;
```

CASE_TYPE	# ROWS_
Active	1398192
Deaths	3837412
Confirmed	4212482
Recovered	290206

Figure 3.2 – Row counts by CASE_TYPE

3.2 Enhancement

The dataset was pivoted and enriched using Python and Snowflake Connector, also joined with a 2023 world population dataset from Kaggle (rajkumarpandey02/2023-world-population-by-country) to compute cases_per_100k and deaths_per_100k, giving population-adjusted metrics that allow fair comparison between small and big countries. The enriched result is stored in a new table, “covid_enriched”, inside a project-owned Snowflake database (“covid_project”), since the Marketplace dataset itself is read-only and cannot be written.

A data quality issue was identified and resolved during step where need to duplicate rows for a small number of pairs, traced to duplicate country entries in the Kaggle population .csv fanning out the merge. This was fixed by deduplicating the population data on the country column before merging, and confirmed resolved via a GROUP BY / HAVING COUNT(*) > 1 verification query returning zero rows.

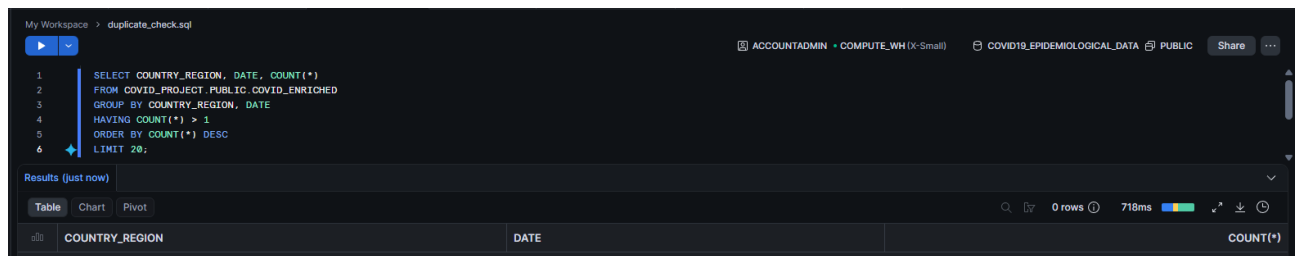


Figure 3.3 – Duplicate-check query on COVID_ENRICHED confirming 0 rows

3.3 Automated EDA

Exploratory data analysis was automated using the ydata-profiling library, generating a full interactive html report (docs / eda_report.html) covering variable distributions, missing-value patterns, correlations, and duplicate detection — without manually writing per-column analysis code.

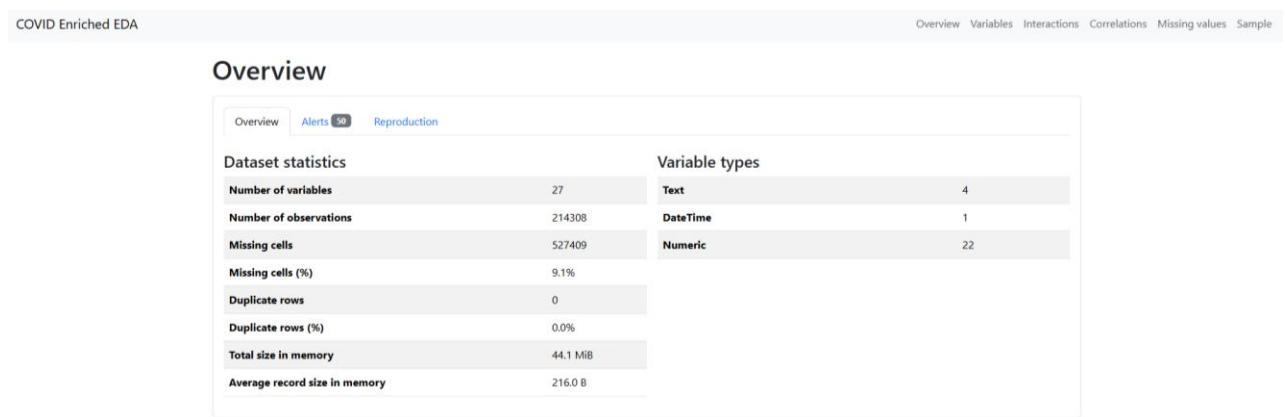


Figure 3.4 – EDA report overview

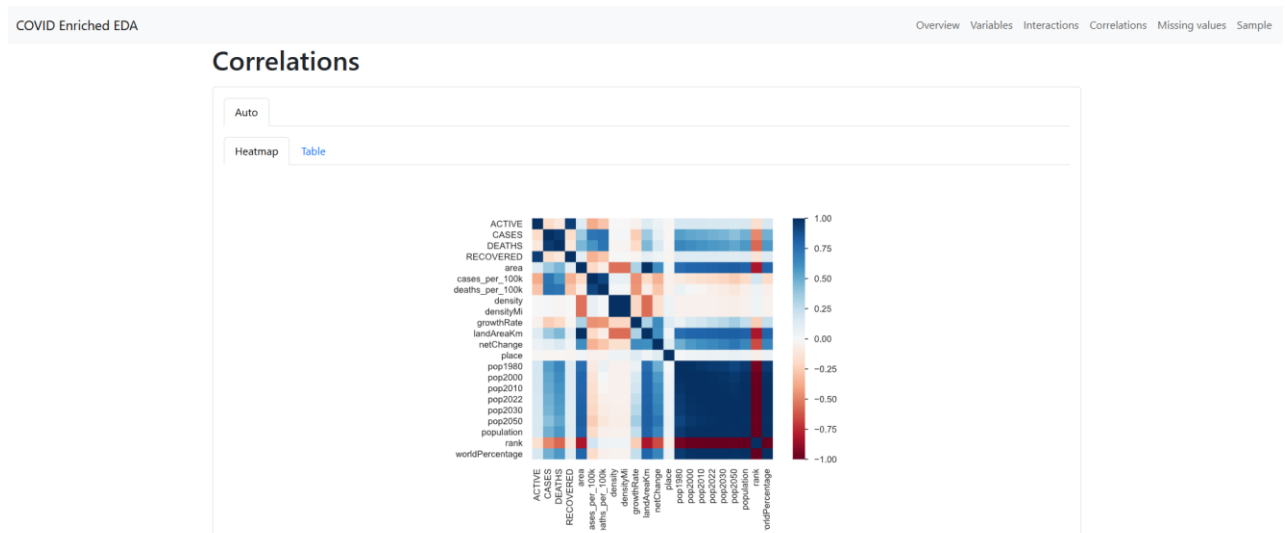


Figure 3.5 – Correlation heatmap across all enriched variables

4. Task 3 – NoSQL Data Modeling

A MongoDB Atlas free cluster M0 tier was used to store semi-structured supplementary data that doesn't fit Snowflake fixed relational schema: user annotations on data points, and user preferences and supplementary sources.

4.1 Schema Design

Collection: annotations

```
{
  "_id": ObjectId,
  "country_region": "Lithuania",
  "date": "2021-04-15",
  "metric": "cases",
  "comment": "Spike due to reporting backlog",
  "author": "user_id_or_name",
  "tags": ["anomaly", "reporting-lag"],
  "created_at": ISODate
}
```

Collection: user_preferences

```
{
  "_id": ObjectId,
  "user_id": "string",
  "default_country": "Lithuania",
  "favorite_metrics": ["cases_per_100k", "deaths_per_100k"],
  "dashboard_theme": "dark"
}
```

Collection: supplementary_sources

```
{
  "_id": ObjectId,
  "country_region": "Lithuania",
  "source_name": "WHO Situation Report",
  "url": "https://...",
  "note": "Alternative case count for cross-checking",
  "added_at": ISODate
}
```

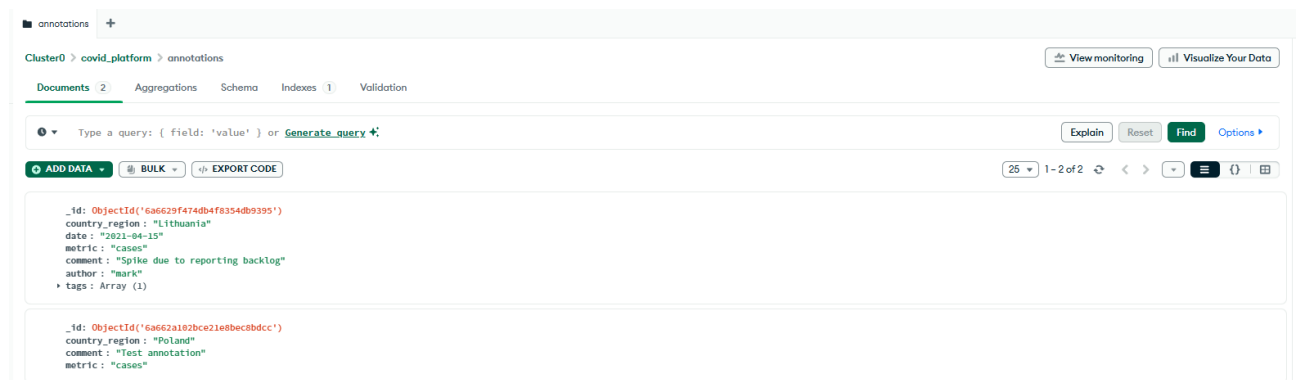


Figure 4.1 – MongoDB Atlas Data Explorer showing the annotations collection, with one document from the seed script and one written live via the dashboard's POST / annotations endpoint

5. Task 4 – API Development with Python

A REST API was built using FastAPI, providing four endpoints:

Endpoint	Method	Description
metrics	GET	Queries Snowflake for enriched COVID metrics by country and date range
forecast	GET	Returns a 30-day case forecast for a country via Prophet
annotations	GET	Fetches saved annotations from MongoDB for a country
annotations	POST	Saves a new annotation to MongoDB

Table 5.1 Endpoints using FastAPI

The API demonstrates integration across both data stores in a single request lifecycle: ‘metrics’ and ‘forecast’ query Snowflake on demand, while ‘annotations’ reads and writes MongoDB – satisfying the requirement to query Snowflake, interact with the NoSQL database, and perform on-the-fly.

COVID-19 Platform API 0.1.0 OAS 3.1

/openapi.json

The image shows a Swagger UI interface for the COVID-19 Platform API. At the top, it says 'default' with an expand/collapse arrow. Below this, there are four endpoint entries, each with a method button, a path, and a description:

- GET** `/metrics` Get Metrics
- GET** `/annotations` Get Annotations
- POST** `/annotations` Add Annotation
- GET** `/forecast` Get Forecast

Below the endpoints, there is a 'Schemas' section with an expand/collapse arrow. It contains two schema entries:

- HTTPValidationError** > Expand all `object`
- ValidationError** > Expand all `object`

Figure 5.1 – Swagger UI showing all four API endpoints

GET

/metrics

Get Metrics

Parameters

Cancel

Name	Description
country * required	
string (query)	Poland
start_date	
string (query)	2020-01-01
end_date	
string (query)	2022-12-31

Execute

Clear

Responses

Curl

curl -X 'GET' \n'http://localhost:8000/metrics?country=Poland&start_date=2020-01-01&end_date=2022-12-31' \n-H 'accept: application/json'

Request URL

http://localhost:8000/metrics?country=Poland&start_date=2020-01-01&end_date=2022-12-31

Server response

Code	Details
200	Response body <pre>[{ "DATE": "2020-01-22", "CASES": 0, "DEATHS": 0, "cases_per_100k": 0, "deaths_per_100k": 0 }, { "DATE": "2020-03-04", "CASES": 1, "DEATHS": 0, "cases_per_100k": 0.0024374746913955947, "deaths_per_100k": 0 }, { "DATE": "2020-03-05", "CASES": 1, "DEATHS": 0, "cases_per_100k": 0.0024374746913955947, "deaths_per_100k": 0 }, { "DATE": "2020-03-06", "CASES": 5, "DEATHS": 0, "cases_per_100k": 0.012187373456977976, "deaths_per_100k": 0 }]</pre> Response headers content-length: 132550 content-type: application/json date: Sun, 26 Jul 2026 15:49:27 GMT server: uvicorn

Figure 5.2 – Successful /metrics response for a Poland

GET

/forecast

Get Forecast

Parameters

Cancel

Name	Description
country * required	
string	
(query)	
periods	
integer	
(query)	

Execute

Clear

Responses

Curl

curl -X 'GET' \n'http://localhost:8000/forecast?country=Poland&periods=30' \n-M 'accept: application/json'

Request URL

http://localhost:8000/forecast?country=Poland&periods=30

Server response

Code	Details
200	Response body <pre>{ "ds": "2023-03-10", "yhat": 6360403.715501143, "yhat_lower": 6333057.607785588, "yhat_upper": 6410214.147928195 }, "ds": "2023-03-11", "yhat": 6360431.922567766, "yhat_lower": 6329130.181163724, "yhat_upper": 6407521.057499994 }, "ds": "2023-03-12", "yhat": 6366820.075056871, "yhat_lower": 6321383.811842929, "yhat_upper": 6406118.245215745 }, "ds": "2023-03-13", "yhat": 6364499.818155621, "yhat_lower": 6323882.372802774, "yhat_upper": 6403779.420786069 }, "ds": "2023-03-14", "yhat": 6365186.401865213, "yhat_lower": 6322806.723800414, "yhat_upper": 6404430.700057070 }</pre> Download Response headers content-length: 3203 content-type: application/json date: Sun, 26 Jul 2026 15:47:23 GMT server: unicorn

Figure 5.3 – Successful /forecast response (Poland, 30-day forecast)

6. Task 5 – Interactive Visualization with Python

A Dash web dashboard was built to visualize infection rates, mortality rates, and a forecast, with a country selector dropdown driving all charts.

- Cases per 100,000 population over time;
- Deaths per 100,000 population over time;
- 30-day forecasted case trajectory (Prophet).



Figure 6.1 – Dashboard showing cases/deaths/forecast charts for Poland

6.1 Bonus: User Annotations

As a bonus feature, the dashboard includes a text box allowing users to add a comment/annotation for the currently selected country, which is saved via the API's POST /annotations endpoint into MongoDB, and existing annotations for that country are displayed back in the dashboard.

Add annotation

Saved!

Figure 6.2 – Annotation saved via the dashboard, with confirmation

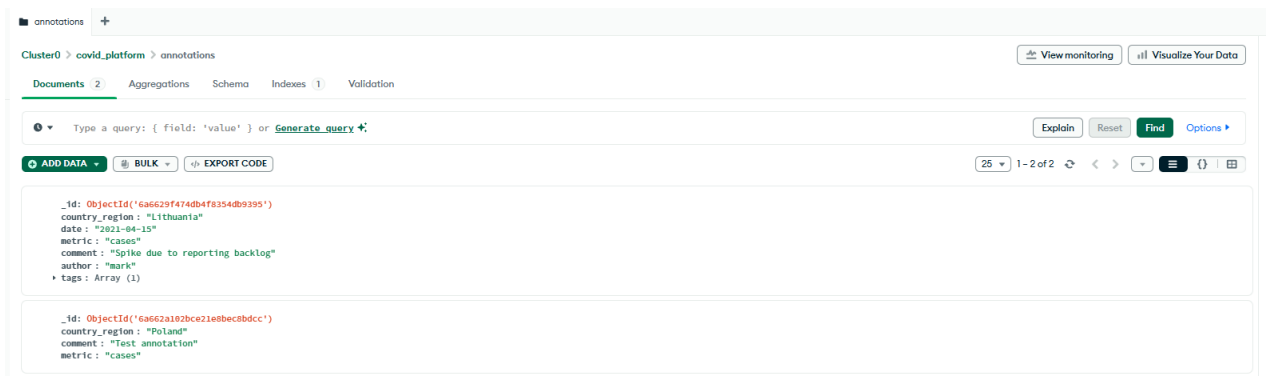


Figure 6.3 – MongoDB Atlas confirming the annotation was persisted

7. Task 6 – Analytical Features

7.1 Time Series Forecasting

Facebook Prophet was used to forecast future cumulative case counts per country, 30 days beyond the available data. Because the underlying CASES metric is a cumulative running total (not new daily cases), the forecast reflects continued cumulative growth rather than a rise/fall infection curve — this is noted explicitly here rather than presented as a naive prediction of new infections.

GET /forecast Get Forecast

Parameters

Cancel

Name	Description
country * required string (query)	Lithuania
periods integer (query)	25

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/forecast?country=Lithuania&periods=25' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/forecast?country=Lithuania&periods=25
```

Server response

Code Details

200

Response body

```
{
  "ds": "2023-03-10",
  "yhat": 1297478.4896187275,
  "yhat_lower": 1290803.8772683673,
  "yhat_upper": 1304978.6121364882,
}
{
  "ds": "2023-03-11",
  "yhat": 1297483.1801033781,
  "yhat_lower": 1289465.8955390272,
  "yhat_upper": 1304983.2312768688,
}
{
  "ds": "2023-03-12",
  "yhat": 1297281.9368559532,
  "yhat_lower": 1289815.6143981362,
  "yhat_upper": 1305436.3909497932,
}
{
  "ds": "2023-03-13",
  "yhat": 1296724.867097017,
  "yhat_lower": 1288916.115237983,
  "yhat_upper": 1304696.0833940995,
}
{
  "ds": "2023-03-14",
  "yhat": 1296944.2472697194,
  "yhat_lower": 1289463.4325989517,
  "yhat_upper": 1304403.2421652268,
}
```

Response headers

```
content-length: 2735
content-type: application/json
date: Sun, 26 Jul 2026 16:03:03 GMT
server: uvicorn
```

Figure 7.2 – Dashboard forecast chart (25-day case forecast, Lithuania)

7.2 Bonus: Clustering

K-Means clustering ($k=4$) was applied to each country's peak cases_per_100k and peak deaths_per_100k to segment countries by pandemic impact pattern, using standardized features. The clustering produced four interpretable groups:

- Cluster A (low cases, low deaths): predominantly African nations and small island/Pacific territories — likely reflecting a mix of genuinely lower transmission and limited testing/reporting infrastructure.

- Cluster B (moderate cases, moderate deaths): a broad, heterogeneous middle group including Sweden, Canada, South Africa, and Japan.
- Cluster C (very high cases, moderate deaths): wealthy, high-testing nations with the highest reported case rates but comparatively controlled mortality — consistent with strong testing infrastructure and earlier vaccine access.
- Cluster D (moderate-high cases, disproportionately high deaths): concentrated in Eastern Europe and Latin America, suggesting case burden translated into disproportionate mortality — plausibly linked to lower vaccination rates and/or healthcare capacity strain during peak waves.

This is the project's most notable insight: the clustering separates countries less by "how much COVID they had" and more by "how deadly it was relative to case volume," with Cluster D standing out as the group where case burden converted into excess mortality most severely.

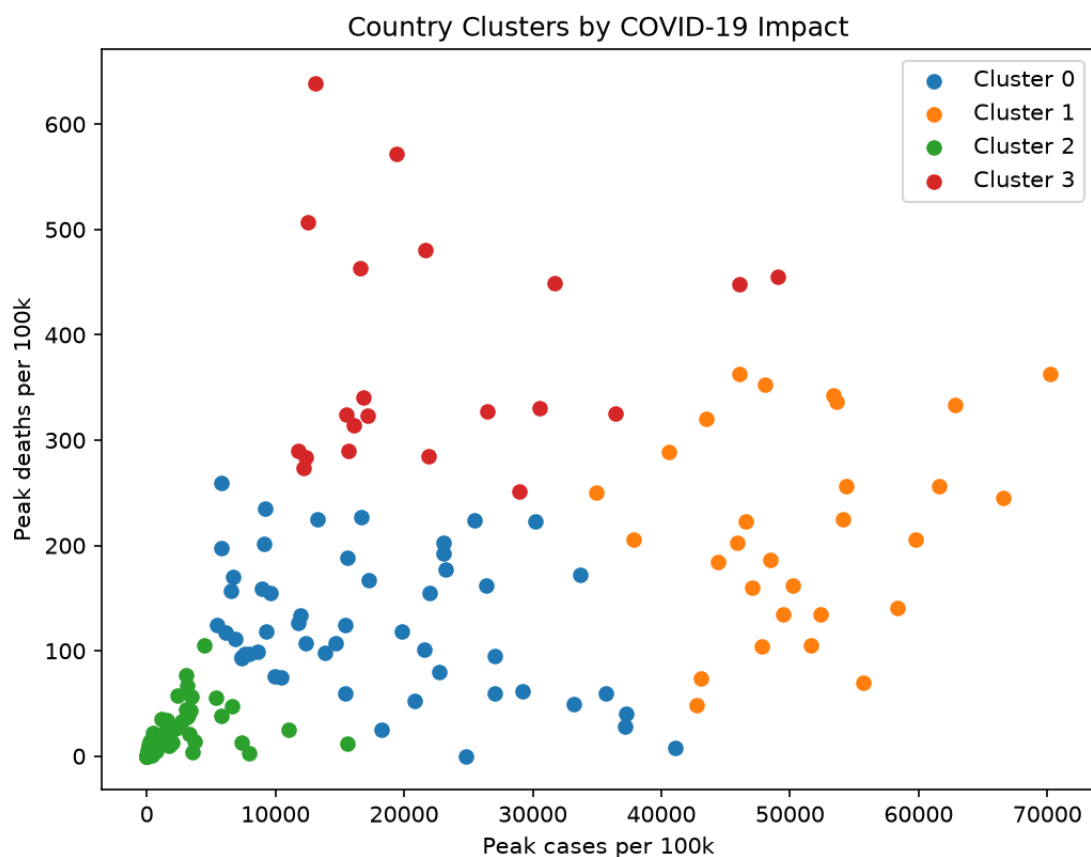


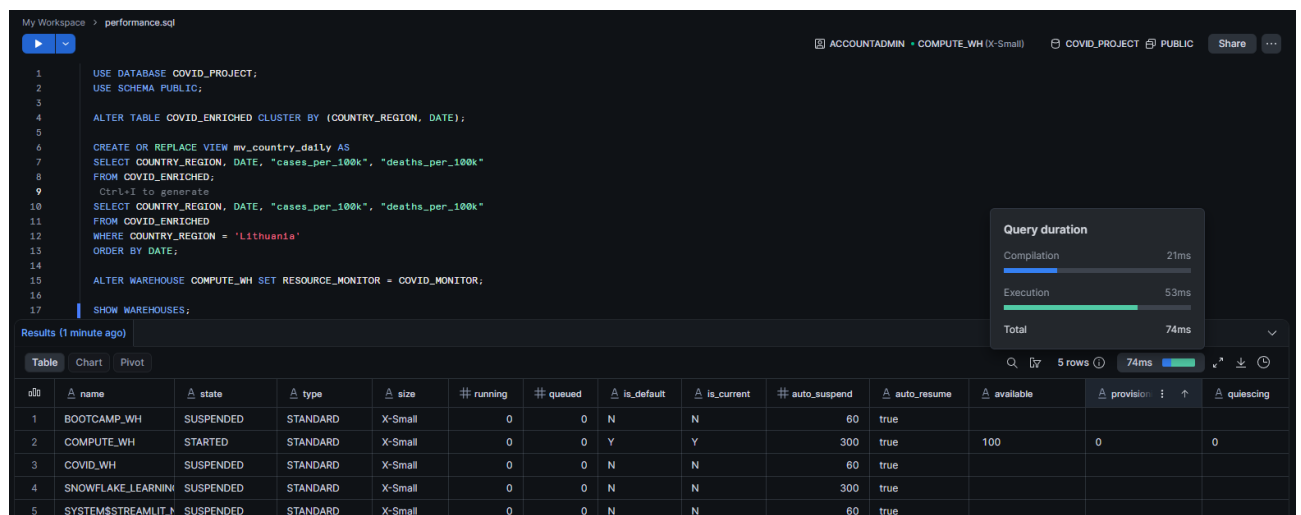
Figure 7.3 – K-Means clustering ($k=4$) of countries by peak cases per 100k vs. peak deaths per 100k. Cluster 3 (red) stands out with moderate case counts but disproportionately high mortality.

8. Task 7 – Performance Optimization

Two Snowflake-native optimizations were applied to ‘covid_enriched’, which is queried repeatedly by country and date range:

- A clustering key on (‘country_region’, ‘date’) was added via ALTER TABLE ... CLUSTER BY, aligning the table's physical micro-partitions with the API's most common filter pattern.
- A view (mv_country_daily) was created to pre-define the common query shape used by the dashboard.

The project also deliberately runs on an X-Small warehouse to conserve trial credits; on the current dataset size, this keeps query costs low, though a larger warehouse would provide more parallel compute for substantially larger data volumes — a straightforward, if costlier, performance lever.



```
1  USE DATABASE COVID_PROJECT;
2  USE SCHEMA PUBLIC;
3
4  ALTER TABLE COVID_ENRICHED CLUSTER BY (COUNTRY_REGION, DATE);
5
6  CREATE OR REPLACE VIEW mv_country_daily AS
7  SELECT COUNTRY_REGION, DATE, "cases_per_100k", "deaths_per_100k"
8  FROM COVID_ENRICHED;
9
10 Ctrl+I to generate
11 SELECT COUNTRY_REGION, DATE, "cases_per_100k", "deaths_per_100k"
12 FROM COVID_ENRICHED
13 WHERE COUNTRY_REGION = 'Lithuania'
14 ORDER BY DATE;
15
16 ALTER WAREHOUSE COMPUTE_WH SET RESOURCE_MONITOR = COVID_MONITOR;
17
18 SHOW WAREHOUSES;
```

Query duration breakdown:

Category	Duration
Compilation	21ms
Execution	53ms
Total	74ms

#	name	state	type	size	# running	# queued	is default	is current	# auto_suspend	auto_resume	available	provision	quiescing
1	BOOTCAMP_WH	SUSPENDED	STANDARD	X-Small	0	0	N	N	60	true		0	0
2	COMPUTE_WH	STARTED	STANDARD	X-Small	0	0	Y	Y	300	true	100	0	0
3	COVID_WH	SUSPENDED	STANDARD	X-Small	0	0	N	N	60	true			
4	SNOWFLAKE_LEARNIN	SUSPENDED	STANDARD	X-Small	0	0	N	N	300	true			
5	SYSTEM\$STREAMLIT	SUSPENDED	STANDARD	X-Small	0	0	N	N	60	true			

Figure 8.1 — SHOW WAREHOUSES confirming COMPUTE_WH as the active warehouse with COVID_MONITOR attached, and query duration breakdown (74ms total: 21ms compilation, 53ms execution) for a filtered COVID_ENRICHED query

9. Task 8 – API Caching

A lightweight in-memory TTL cache was implemented as a Python decorator and applied to the ‘metrics’ (5-minute TTL) and ‘forecast’ (10-minute TTL) endpoints, avoiding redundant Snowflake queries and Prophet model refits for repeated identical requests within the cache window.

```
@app.get("/metrics")
@cache(ttl=300)
def get_metrics(country: str = Query(...), ...):
    ...
```


10. Task 9 – Pattern Recognition (MATCH_RECOGNIZE)

Snowflake's MATCH_RECOGNIZE clause was used to detect temporal patterns directly in SQL, without exporting data to Python.

10.1 Cumulative Rise Streaks

An initial query searched for 3+ consecutive days of rising cumulative case counts per country. As expected given that CASES is a cumulative running total, this returned one very long streak per country (the longest being Iran at 824 consecutive days) – a useful demonstration of MATCH_RECOGNIZE mechanics, but not itself an epidemiologically meaningful pattern, since a cumulative total rarely decreases.

10.2 Accelerating New-Case Bursts

A second, more meaningful query computed day-over-day new cases and used 'match_recognize' to find 3+ consecutive days of accelerating new-case growth — genuine "wave onset" signals. This surfaced clear, interpretable results:

- Egypt showed the longest sustained acceleration phase of any country: 49 consecutive days.
- Russia appeared twice among the longest streaks: 27 days in January 2022, coinciding with the global Omicron wave onset, and 23 days in September–October 2020, aligning with a known second-wave period.
- Several Gulf states showed short but sharp acceleration bursts, consistent with dense urban populations enabling fast transmission spikes.

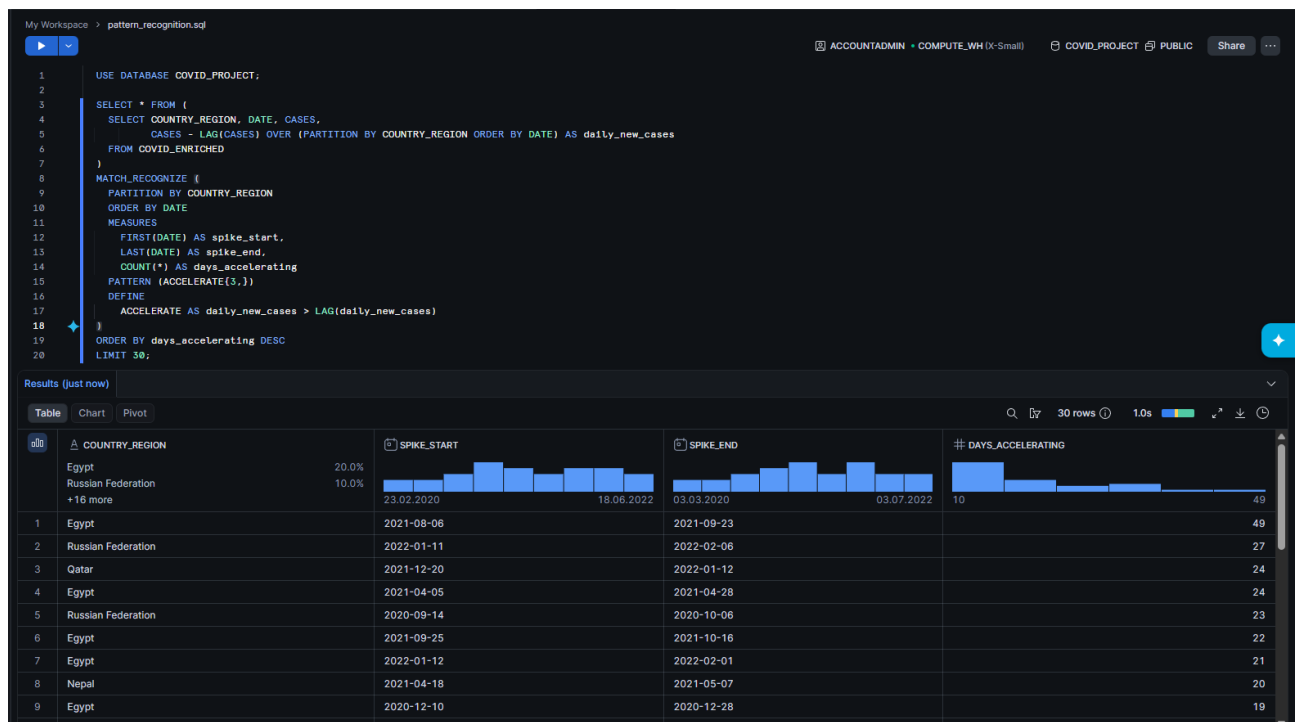


Figure 10.1 – MATCH_RECOGNIZE on accelerating new-case bursts: Egypt (49 days), Russia (27 days, Omicron wave)

My Workspace > match_recognize.sql

ACCOUNTADMIN • COMPUTE_WH (X-Small) COVID_PROJECT PUBLIC Share

```

1  USE DATABASE COVID_PROJECT;
2
3  SELECT * FROM (
4    SELECT COUNTRY_REGION, DATE, CASES FROM COVID_ENRICHED
5  )
6  MATCH_RECOGNIZE (
7    PARTITION BY COUNTRY_REGION
8    ORDER BY DATE
9    MEASURES
10     FIRST(DATE) AS spike_start,
11     LAST(DATE) AS spike_end,
12     COUNT(*) AS days_in_spike
13     PATTERN (RISE{3,})
14     DEFINE
15     RISE AS CASES > LAG(CASES)
16   )
17   ORDER BY days_in_spike DESC;

```

Results (just now)

Table Chart Pivot 6,055 rows 555ms

	COUNTRY_REGION	SPIKE_START	SPIKE_END	DAYS_IN_SPIKE
1	Iran, Islamic Republic of	2020-02-19	2022-05-22	824
2	United States	2020-02-29	2022-05-22	814
3	Austria	2020-02-29	2022-05-20	812
4	Poland	2020-03-08	2022-05-22	808
5	Morocco	2020-03-10	2022-05-22	804
6	Russian Federation	2020-03-12	2022-05-22	802
7	Malaysia	2020-03-13	2022-05-22	801
8	Turkey	2020-03-15	2022-05-19	798
9	Brazil	2020-03-15	2022-05-16	793
10	Lithuania	2020-06-30	2022-08-27	789
11	Korea, Republic of	2020-02-20	2022-04-12	783
12	Canada	2020-02-27	2022-04-07	771
13	Ukraine	2020-03-24	2022-05-02	770
14	Iraq	2020-03-13	2022-04-12	761

Figure 10.2 – MATCH_RECOGNIZE on cumulative totals: 6,055 qualifying streaks found, longest being Iran at 824 days

11. Task 10 – Source Code and Repository

All source code, SQL scripts, and configuration are version-controlled and available at:

<https://github.com/MarkTretiak/covid19-platform.git>

The repository README.md contains complete setup instructions enabling the project to be deployed and run on any machine, not only the development laptop, including environment variable configuration, Snowflake/MongoDB setup steps, and run commands for the API and dashboard.

12. Key Insights

- The clustering result is probably the most interesting finding in this project: the split between countries isn't really about total case volume, it's about how much of that case volume turned into deaths. Eastern Europe and Latin America stand out as the group where that ratio was worst.
- Reported case rate looks like it tracks testing capacity almost as much as actual spread – nearly every country in the highest cases-per-100k cluster is a wealthy, high-testing country, while several lower-income countries show suspiciously low numbers on both cases and deaths, which is more likely a reporting gap than genuinely low transmission.
- MATCH_RECOGNIZE was able to pick out a known real-world event (the Omicron wave hitting Russia in January 2022) purely from the shape of the data, with no external labeling, which was a good confirmation that the pattern-matching approach actually works and isn't just a syntax exercise.
- A few non-country entries in the dataset, such as “Winter Olympics 2022” and “Summer Olympics 2020,” represent event venues rather than nations. These show normal case growth during the event itself, then a flat, unchanging count afterward, which indicates reporting stopped once the event ended rather than an ongoing data problem.

13. Limitations

- Country name matching between Snowflake and the Kaggle population data isn't perfect, so a number of population-adjusted metrics come out blank for smaller or oddly-named territories.
- A materialized view and a shared cache (Redis) would both be better production choices than what was used here, a regular view and an in-memory cache, but weren't necessary at this project's scale.

- The forecast is built on cumulative cases rather than new daily cases, which makes it less useful as an actual “is this getting worse” signal than it could be.
- The API interpolates the country parameter directly into the SQL string rather than using parameterized queries, which should be fixed before this went anywhere near production.

14. Conclusion

This project ended up being a good way to actually see how a full data stack fits together instead of just reading about each piece separately: Snowflake, MongoDB, an API layer, and a dashboard, plus forecasting, clustering, and MATCH_RECOGNIZE. Most of the real learning came from debugging things that did not work on the first try, such as the dataset being pivoted differently than expected, the duplicate rows in the enrichment step, and materialized views not being available on a Standard-edition trial, rather than from the parts that went smoothly. The clustering result in particular, showing that case burden and mortality are not the same story, was the most genuinely interesting outcome of the whole project.